

Title: Predictive Insight Engine

Duration: 6 Hours

Objective

The objective of this project is to help students understand, implement, and evaluate supervised learning algorithms, with a strong emphasis on regression techniques and model performance analysis. Students will explore linear regression variants, gradient descent optimization, and model behavior concepts such as bias–variance trade-off, overfitting, and underfitting.

Problem Statement

You are working as a **Junior Data Scientist** at a real estate analytics firm. The company wants to build a **predictive system** to estimate **house prices** based on various features such as size, number of rooms, location score, and age of the property.

Your task is to **design, implement, analyze, and evaluate multiple supervised learning models** to predict house prices accurately and to explain **why certain models perform better than others**.

Part A: Conceptual Understanding (Theory)

Answer the following in brief (2–4 lines each):

1. What are **Supervised Learning Algorithms**?
2. Difference between **Regression Algorithms vs Classification Algorithms**.
3. Explain **Simple Linear Regression**.
4. List and explain the **Assumptions of Linear Regression**.
5. What is **Bias–Variance Trade-Off**?

6. Explain **Overfitting** and **Underfitting** with examples.
-



Part B: Dataset Understanding & Preparation

You are provided with a dataset containing the following features:

- House Area (sq.ft)
- Number of Bedrooms
- Number of Bathrooms
- Location Score
- Age of Property
- House Price (Target Variable)

Dataset: [Access from here](#)

Tasks:

7. Identify **independent** and **dependent variables**.
 8. Visualize relationships between features and target variable.
 9. Split the dataset into **training and testing sets**.
-



Part C: Simple Linear Regression

10. Implement **Simple Linear Regression** using one feature (e.g., House Area).
 11. Plot the **regression line** and interpret the slope and intercept.
 12. Validate **linear regression assumptions** using plots and observations.
-



Part D: Model Evaluation Metrics

13. Evaluate the Simple Linear Regression model using:
 - Mean Squared Error (MSE)
 - Mean Absolute Error (MAE)
 - Root Mean Squared Error (RMSE)
 - R^2 Score
 - Adjusted R^2 Score

14. Interpret each metric and explain what it reveals about model performance.



Part E: Multiple Linear Regression

15. Implement **Multiple Linear Regression** using all relevant features.
 16. Compare its performance with Simple Linear Regression.
 17. Explain why performance improves or degrades.
-



Part F: Polynomial Regression

18. Implement **Polynomial Regression** (degree 2 or 3).
 19. Compare linear vs polynomial regression visually and numerically.
 20. Identify signs of **overfitting** or **underfitting**.
-



Part G: Gradient Descent Optimization

21. Explain **Gradient Descent** conceptually.
 22. Implement **Batch Gradient Descent** from scratch.
 23. Implement **Stochastic Gradient Descent (SGD)**.
 24. Implement **Mini-Batch Gradient Descent**.
 25. Compare convergence behavior and training time of all three methods.
-



Part H: Bias–Variance & Model Diagnostics

26. Analyze bias and variance across:
 - Simple Linear Regression
 - Multiple Linear Regression
 - Polynomial Regression
 27. Explain how model complexity affects prediction error.
 28. Identify which model best balances **bias and variance**.
-



Part I: Final Analysis & Reporting

29. Summarize findings in a short report covering:
- Best-performing model and why
 - Impact of gradient descent optimization
 - Evidence of overfitting/underfitting
 - Practical business interpretation of results
30. Submit:
- Source code
 - Model evaluation results
 - Graphs and plots
 - Final conclusion



Submission Guidelines

- Include practical implementation in a Jupyter Notebook and screenshots.
- Label all charts clearly and write short interpretations under each result.
- **GitHub Repository:**
 - Create a GitHub repository to host your project.
 - Upload your project files, including source code, and documentation to the repository.
 - Add a document (PDF) explaining theory concepts with definitions (if necessary).
 - Submit your video link only via a Google Drive sharable link or via a YouTube unlisted link.
 - Ensure that you provide a clear and descriptive README.md file.

Marking Criteria (10 Marks)

Total Marks: 10 — distributed across 3 components as follows:

Component	Marks	Weight	Deliverable
A. Technical Completion (Jupyter Notebook .ipynb)	5 Marks	50%	Completed notebook — all tasks, Restart & Run All clean
B. Recorded Video Explanation (Face + Screen)	3 Marks	30%	5–10 min MP4/MOV: face + screen, verbal concept explanations

C. GitHub Repository + README.md	2 Marks	20%	Public repo with .ipynb + screenshots + README + video link
TOTAL	10 Marks	100%	

Component B: Recorded Video — Face + Screen (3 Marks)

Record a video showing your face (webcam picture-in-picture) and your Jupyter Notebook screen simultaneously. Explain concepts verbally throughout — do not just execute cells without speaking.

- **You can use OBS Studio software for recording.**
- **Submit your video link only via a Google Drive sharable link or via a YouTube unlisted link.**

Remember to follow the instructions provided professionally, make suitable assumptions wherever necessary, and avoid copying code or content from unauthorized sources. Good luck with your project work!

Predictive Insight Engine
Supervised Learning

BRING ON YOUR CODING ATTITUDE